

层级	硬件抽象	单核单线程模型程序员视角	单核多线程模型程序员视角
执行模型	考虑维度	- 是否暴露线程 - vector / CUBE 对外呈现	
	PE / thread	不可见	通过 thread_id 显式映射 4 个 PE，每个线程实例只代表当前 PE。
	cube	tmatmul 调用。	gmma(tA, tBReuse, tC) 中 tA/tC 是当前 PE 分片，硬件把 4 个 PE 的分片逻辑合成 A_big/C_big；mma 调用。
	vector	tileop	tileop
内存模型	考虑维度	- cell / staging B 是否对外可见 - staging B 是 reg 还是 buffer；如果是 buffer，需要程序员管理内存 - layout 转换是否显式需要：cube 需要 Nz/Zn，vector 需要 ND/DN - 寄存器对外呈现（缺乏资料）	
	PE-local cell	逻辑构成完整大 tileA [tM,tK]、tileC [tM,tN]。	线程控制当前 PE 的 tileA [tM/thread_num.tK]、tileC [tM/thread_num.tN]；其它 PE 的 tile 不可见。
	staging B	不可见	B 不按 PE 切分；以 tileB [tK,tN] / tBReuse 进入 shared staging B，被多个 PE 和多个 M block 复用。
	global memory	global_iterator 直接索引完整 tile，例如 gAlter(i,k)、gCIter(i,j)。	根据 thread_id 手动切分 M 轴 tile，如 gAlter(i*thread_num+thread_id.k)。
	layout 转换	不可见。	ND2Nz/ND2Zn 等 layout 由 GMMMA/cube 处理，不可见。
模型对比	code example	<pre>matmul_gmma_ReuseB(){ using tileA = TileLeft<dtype, tM, tK>; // B is shared, not split: using tileB = TileRight<dtype, tK, tN>; using tileC = TileAcc<float, tM, tN>; tileB tBReuse; // B load outside N loop, reused by all j/i blocks TLOAD(tBReuse, gBIter()); for (int j = 0; j < Nb; ++j) { for (int i = 0; i < Mb; ++i) { tileC tAcc; tileA tA; // compiler/hw computes 4 PE base addresses by tileA size TLOAD(tA, gAIter(i)); // lhs input: tA[tM,tK] // rhs input: cached B_shared [tK,tN] tmatmul(tA, tBReuse, tAcc); // tAcc is ND layout, 4PE group vector op trowmax(tAcc); // compiler/hw inserts 4 store ops TSTORE(gC, tileC); } } } int main(){ // M-split kernel, single-thread model, Janus-like matmul_gmma_ReuseB<dtype, M, N, K, tileM, tileN, tileK>{ dstC, srcA, srcB; } }</pre>	<pre>// thread kernel matmul_gmma_ReuseB(){ thread_id = get_thread_id(); using tileA = TileLeft<dtype, tM / thread_num, tK>; // B is shared, not split: using tileB = TileRight<dtype, tK, tN>; using tileC = TileAcc<float, tM / thread_num, tN>; tileB tBReuse; // load into staging B; current thread issues group GMMMA.ld TLOAD(tBReuse, gBIter()); for (int j = 0; j < Nb; ++j) { for (int i = 0; i < Mb; ++i) { tileC tAcc; tileA tA; // each PE independently loads tile A shard into cell TLOAD(tA, gAIter(i * thread_num + thread_id)); // Collective GMMMA with reused B: // lhs input: tA[B..3] -> A_big [tM,tK] // rhs input: tBReuse[k] -> B_shared [tK,tN] // dst: tAcc -> C_pe [tM/kPeNum,tN] // Hopper-like: each thread issues one gmma instruction, // A address points to PE-local cell; Sb points to shared staging B. // gmma internally organizes cube ND2Nz and ND2Zn layout conversion. gmma(tA, tBReuse, tAcc); TSTORE(gC, tileC); } } } int main(){ // M-split kernel, launch thread_num threads, Hopper-like matmul_gmma_ReuseB<<<thread_num>>>><dtype, M, N, K, tileM, tileN, tileK>{ dstC, srcA, srcB, thread_num; } }</pre>
	优劣势分析	- 优点：编程直观简单，纯api编程，不感知gmma和mma差异。 - 缺点：无法表达PE间多个小tile的矩阵计算并行，完全由硬件/编译器控制，负担较重。	- 优点：控制力度更灵活。可以做更细粒度的流水编排 - 缺点：尾块处理多线程编程可能比较复杂。
	算子实现	可直接复用现有一层架构的代码	✔ matmul ✔ fa
	遗留问题	-	gmma 限定 4 PE 锁步，理论上 thread 也只能是 4，thread 无法脱离 PE 做抽象
TODO		- 生态兼容问题 - NVIDIA 代码迁移是否方便 - warp specialization 是否需要，如何映射	